Programming paradigms

Introduction
Syntax and Semantics
Coding Standards and Conventions

Course Objectives

- *Not* to teach specific programming languages. Haskell, C++ and Prolog languages are tools of this course.
- Studying the common concepts of programming languages.
- Studying the different paradigms and approaches.
- What is the measure of quality in a programming language?
- Construct a basis for other topics like compiler design, software engineering, object oriented design, human computer interaction...

Main books

- C. Ghezzi and M. Jazayeri. *Programming language concepts* (3RD EDITION). John Wiley & Sons, 1996
- J. C. Mitchell. *Concepts in programming languages*. Cambridge University Press, 2003.

Goals

- Critical *thinking* about programming languages.
  - What is a programming language!?
  - Who defines it?
  - How can it be changed?
- *Study* programming languages.
  - Be familiar with basic language *concepts*.
  - Appreciate trade-offs in language *design*.
- Trace *history*, appreciate *evolution* and diversity of *ideas*.
- Be prepared for new programming *methods*, *paradigms*.

Why study programming languages?

- To improve the ability to develop effective algorithms.
- To improve the use of familiar languages.
- To increase the vocabulary of useful programming constructs.
- To allow a better choice of programming language.
- To make it easier to learn a new language.
- To make it easier to design a new language.
- To simulate useful features in languages that lack them.
- To make better use of language technology wherever it appears.

What makes a good language?

- Clarity, simplicity, and unity.
- Orthogonality.
- Naturalness for the application.
- Support of abstraction.
- Ease of program verification.
- Programming environments.
- Portability of programs.
- Cost of use.
  - Cost of execution.
  - Cost of program translation.
  - Cost of program creation, testing, and use.
  - Cost of program maintenance.

What makes a language successful?

- Expressive power.
- Ease of use for the novice.
- Ease of implementation.
- Standardisation.
- Many useful libraries.
- Excellent compilers (including open-source)
- Economics, patronage, and inertia.

Note the recent trend of big companies to create/control their own languages: C# (Microsoft), Hack (Facebook), Go (Google), Objective-C/Swift (Apple), Rust (Mozilla) and perhaps even Python (Dropbox hired Guido van Rossum).

? Why are there so many languages?

- Evolution.
- Special purposes.
- No one language is good at expressing all programming styles.
- Personal preference.

? What makes languages evolve?

- Changes in hardware or implementation platform
- Changes in attitudes to safety and risk
- New ideas from academic or industry

Motivating purpose and language design

A specific purpose or motivating application provides *focus* for language design - what features to include and (harder!) what to leave out. E.g.

- Lisp: symbolic computation, automated reasoning
- FP: functional programming, algebraic laws
- BCPL: compiler writing
- Simula: simulation
- C: systems programming [Unix]
- ML: theorem proving
- Smalltalk: Dynabook [1970-era tablet computer]
- Clu, SML Modules: modular programming
- C++: object orientation
- Java, JavaScript: Internet applications

Program execution model

Good language design presents *abstract machine*.

- Fortran: Flat register machine; memory arranged as linear array
- Lisp: cons cells, read-eval-print loop
- Algol family: stack of activation records; heap storage
- BCPL, C: underlying machine + abstractions
- Simula: Object references
- FP, ML: functions are basic control structure
- Smalltalk: objects and methods, communicating by messages
- Java: Java virtual machine

Classification of programming languages

See `en.wikipedia.org/wiki/Programming_paradigm` for more detail:

- Imperative

procedural	C, Ada, Pascal, Algol, Fortran, . . .
object-oriented	Scala, C#,Java, Smalltalk, SIMULA, . . .
scripting	Perl, Python, PHP, JavaScript, . . .

- Declarative

functional	Haskell, SML, Lisp, Scheme, . . .
logic	Prolog
dataflow	Id, Val
constraint-based	spreadsheets
template-based	XSLT

Language standardisation

Consider: `int i; i = (1 && 2) + 3 ;`

? Is it valid C code? If so, what's the value of `i`?

? How do we answer such questions!?

! Read the reference manual (ISO C Standard).

! Try it and see!

Other languages may have *informal* standards (defined by a particular implementation but what do we do if the implementation is improved?) or *proprietary* standards.

Language-standards issues

Timeliness. When do we standardise a language?

Conformance. What does it mean for a program to adhere to a standard and for a compiler to compile a standard?

Ambiguity and freedom to optimise – Machine dependence – Undefined behaviour.

A language standard is a treaty setting out the rights and obligations of the programmer and the implementer.

Obsolescence. When does a standard age and how does it get modified?

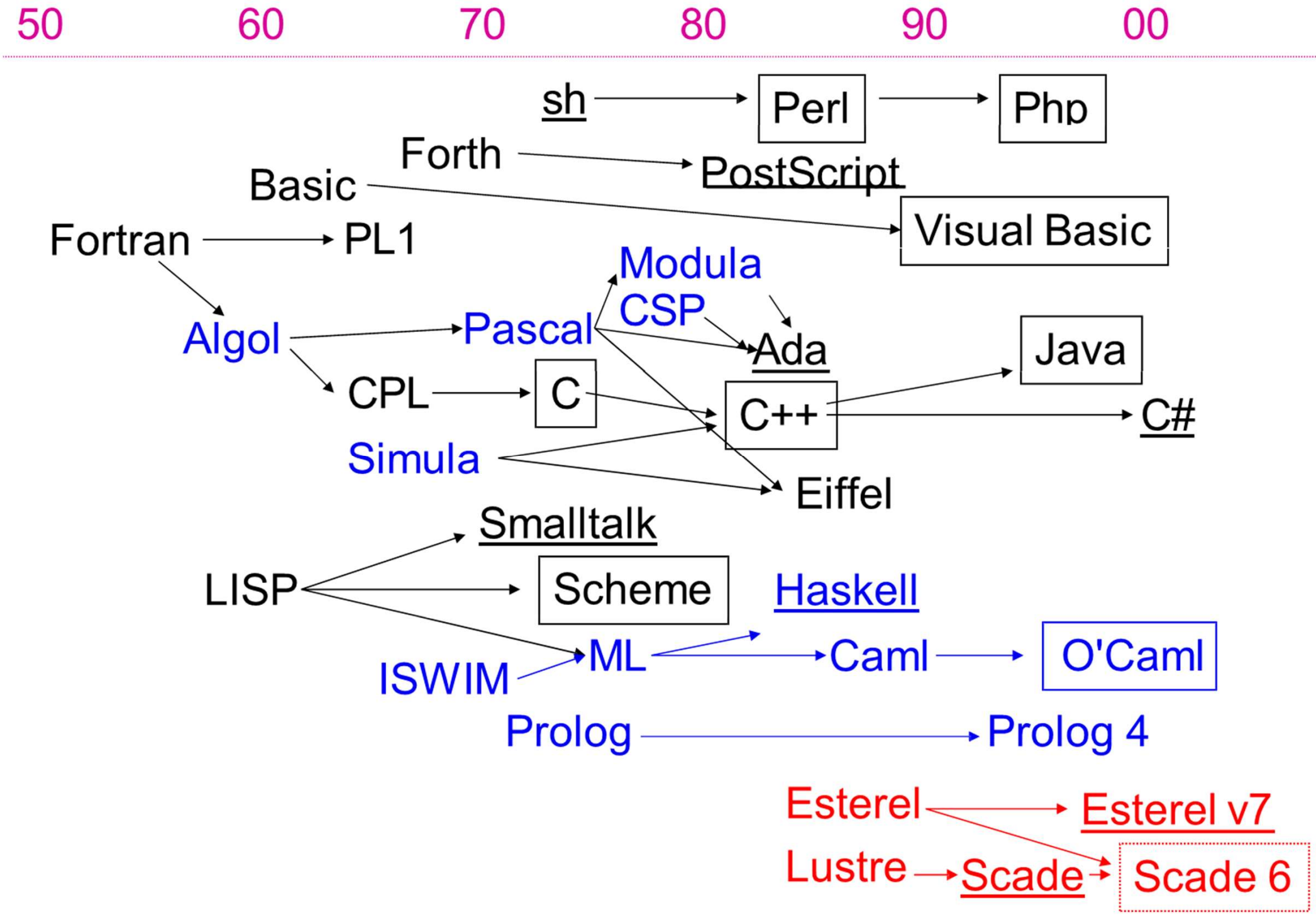
Deprecated features.

Ultra-brief history

1951–55: Experimental use of expression compilers.
1956–60: **Fortran**, COBOL, **Lisp**, **Algol** 60.
1961–65: APL notation, Algol 60 (revised), SNOBOL, CPL.
1966–70: APL, SNOBOL 4, Fortran 66, BASIC, **SIMULA**, Algol 68, Algol-W, BCPL.
1971–75: **Pascal**, PL/1 (Standard), C, Scheme, Prolog.
1976–80: **Smalltalk**, Ada, Fortran 77, ML.
1981–85: Smalltalk-80, Prolog, Ada 83.
1986–90: C++, **SML**, Haskell.
1991–95: Ada 95, TCL, Perl.
1996–2000: Java, JavaScript
2000–05: C#, Python, Ruby, Scala.
1990– : Open/MP, MPI, Posix threads, Erlang, X10 MapReduce, Java 8 features.

For more information:

`en.wikipedia.org/wiki/History_of_programming_languages`


50
60
70
80
90
00
sh
Perl
Php
Forth
PostScript
Basic
Visual Basic
Fortran
PL1
Modula
CSP
Algol
Pascal
Ada
Java
CPL
C
C++
C#
Simula
Eiffel
Smalltalk
LISP
Scheme
Haskell
ISWIM
ML
Caml
O'Caml
Prolog
Prolog 4
Esterel
Esterel v7
Lustre
Scade
Scade 6

Syntax and Semantics

- Syntax: the form and structure of a program.
- Semantics: meaning of a program
- Language definitions are used by:
  - Programmers
  - Implementors of the language processors
  - Language designers

Definitions

- A sentence is a string of characters over some alphabet
- A language is a set of sentences
- A lexeme is the lowest level syntactic unit of the language (i.e.
- `++, int, total`)
- A token is a category of lexemes (i.e. *identifier*)
- syntax recognition: read input strings of the language and verify the input belonging to the language
- syntax generation: generate sentences of the language (i.e. from a given data structure)
- Compilers and interpreters recognize syntax and convert it into machine understandable form.

Definitions

identifiers: x, fact, toto, foo, WikiPedia, ...

numbers: 1, 3.14, 6.023 E 23

keywords: int. for. while. fun. let. rec. emit. ..

```
int i, j ;
int fact (int n) { ... }
let rec fact n = ...
```

declarations

```
x := 1 ;
for (i=0 ; i<N ; i++) { ... }
return 3*x + fact(i) ;
```

instructions

expressions

Striking style: C, C++, Java, ...

```
int fact (int n) {
   int r = 1, i;
   for (i = 2; i <= n; i++) {
      r = r*i;
   }
   return r;
}
```

Round style: Lisp, Scheme

```
(de fact (n)
   (if (= n 1)
      1
       (* n (fact (- n 1)))))
```

```
(de map (f l)
   (if (null l)
       nil
       (cons (f (car l)) (map f (cdr l)))))
```

Mathematical style: ML, Haskell, ...

```
let rec fact n =
  if n=1 then 1 else n* fact (n-1)
in fact 4;;
- : int = 24
```

```
let rec map f list =
  match list with
     [] -> []
  |  head :: tail -> (f head) :: (map f tail);;
val map : ('a -> 'b) -> 'a list -> 'b list = <fun>
```

Logical style: Prolog

```
Pere (Gerard, Antoine).
Pere (Robert, Gerard).
Pere (x,y), Pere (y,z) :- GrandPere (x,z).

?- GrandPere (GP, Antoine).

> GP = Robert
```

Verbal style: Ada, Esterel,...

```
abort
   run  SLOWLY
when 50 Meter;
abort
   every Step do
      emit Jump || emit Breathe
   end every
when 15 Second
```

Backus-Naur Form - Formal grammar

- Programming languages are usually in context free language class
- BNF introduced by John Bakus and modified by Peter Naur for describing Algol language
- It is a meta-language that decribes other languages
- Extended BNF improves readability of BNF

Backus-Naur Form - Formal grammar

```
Expression ::= number
             | ident
             | Expression op Expression
             | '(' Expression ')'
             | ident '(' Expression_List ')';
Expression_List : = Expression
                  | Expression_List ',' Expression;
```

And disambiguation by priorities:

a + b / c $\rightarrow$ a + (b / c) and not (a + b)/ c

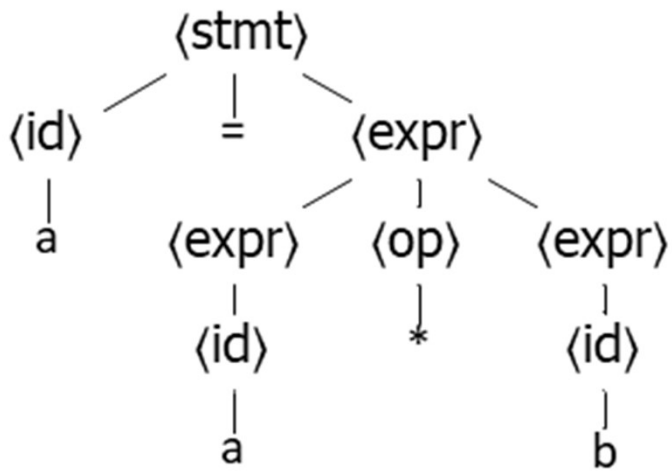
The grammar itself is written in a programming language!

Parse Tree

- Steps of a derivation gives the structure of the sentence. This structure can be represented as a tree.
- All non-terminals used in derivation are intermediate nodes. Each grammar rule replaces the non-terminal node with is children. Root node is the start symbol.
- Terminal nodes are the leaf nodes.
- preorder traversal of leaf nodes gives the resulting sentence.
- leftmost and rightmos derivations can be retrieved by traversal of the tree.

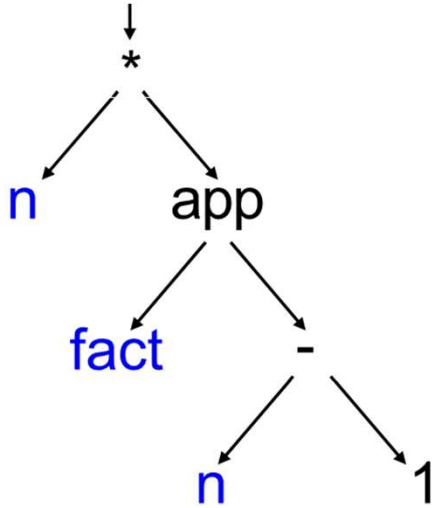
Parse Tree Example

a = a * b



C, CAML : n * fact (n-1)

Lisp, Scheme : (* n (fact (- n 1))



Coding Standards and Conventions

- **Quy chuẩn đặt tên (Naming Conventions):**
  - **camelCase:** Chữ cái đầu viết thường, các từ tiếp theo viết hoa chữ cái đầu ($thisIsMyVariable). Thường dùng cho biến, hàm.
  - **PascalCase:** Viết hoa chữ cái đầu của mỗi từ ($ThisIsMyVariable). Thường dùng cho Class, Interface.
  - **snake_case (underscore):** Dùng dấu gạch dưới (_) để phân cách các từ, viết thường toàn bộ ($this_is_my_variable). Thường dùng trong Python, PHP, biến cục bộ.
  - **SCREAMING_SNAKE_CASE:** Viết hoa toàn bộ và dùng gạch dưới, thường dùng cho hằng số (CONSTANT_NAME).

Coding Standards and Conventions

- **Quy ước về Cấu trúc và Định dạng:**
  - **Thụt lề (Indentation):** Sử dụng 2 hoặc 4 spaces/tab để phân cấp các khối mã bên trong câu lệnh điều kiện, vòng lặp.
  - **Dấu ngoặc nhọn {}:** Quy ước đặt dấu ngoặc ở cuối dòng (K&R style) hoặc xuống dòng mới (Allman style).
  - **Độ dài dòng:** Hạn chế độ dài mỗi dòng (thường dưới 80-120 ký tự) để dễ đọc mà không cần cuộn ngang.
  - **Dòng trống:** Sử dụng dòng trống để phân cách các logic khác nhau trong một hàm hoặc giữa các hàm.
- **Best Practices:**
  - Tránh viết hàm quá dài (nên chia nhỏ).
  - Tránh hard-code giá trị.
  - Xử lý ngoại lệ (Exception Handling) cẩn thận.

Coding Standards and Conventions

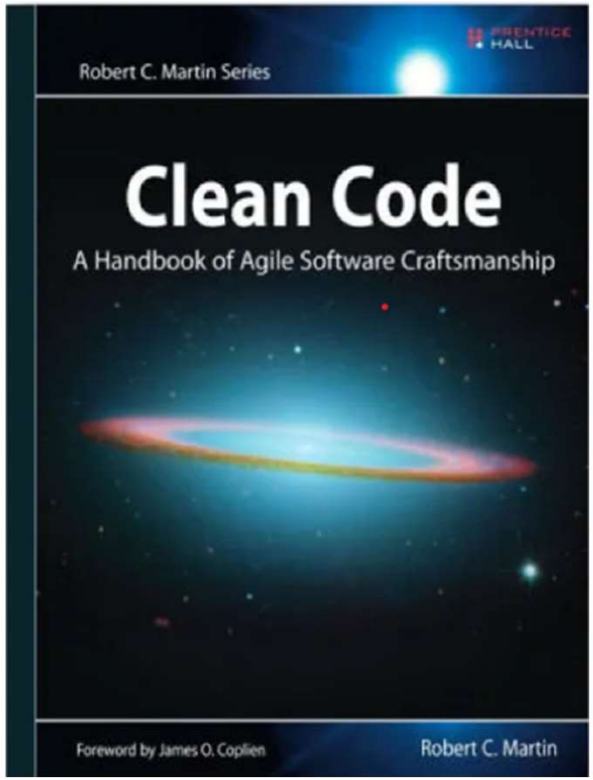
- **Tiền tố (Prefixes) trong đặt tên:**
  - **Biến toàn cục:** Thường có tiền tố g_.
  - **Biến con trỏ:** Thường có tiền tố p.
  - **Biến tĩnh:** Thường có tiền tố s.
- **Tại sao cần chuẩn lập trình?**
  - **Tính nhất quán:** Giúp mã nguồn đồng bộ, dù được viết bởi nhiều lập trình viên.
  - **Dễ đọc và hiểu:** Giúp người khác (hoặc chính bạn trong tương lai) nhanh chóng nắm bắt code.
  - **Dễ bảo trì:** Dễ dàng hiểu và sửa lỗi code sau một thời gian dài.
  - **Chuyên nghiệp:** Nâng cao chất lượng dự án.

Các dự án thường tuân theo các hướng dẫn cụ thể như:

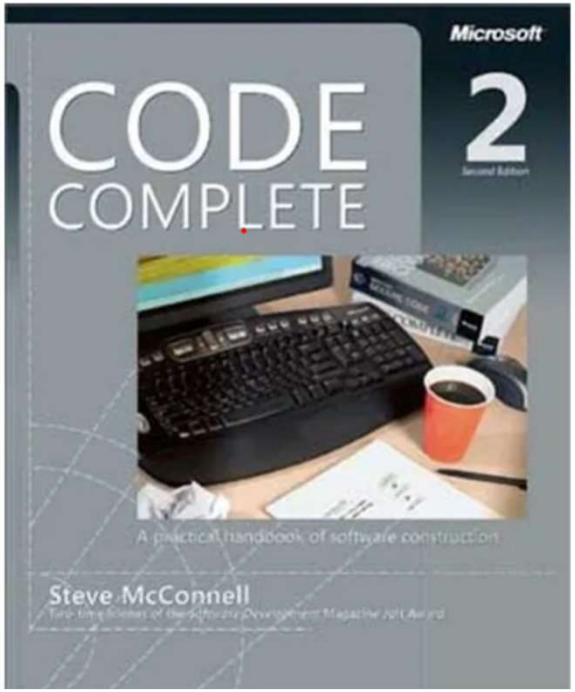
- PEP 8 **Python**
- Airbnb **JavaScript** Style Guide
- **Java:** Google Java Style Guide
- Hoặc chuẩn của từng ngôn ngữ/framework.

Coding Standards and Conventions

- Clean Code. A Handbook of Agile Software Craftsmanship by Robert C. Martin



- Code Complete: A Practical Handbook of Software Construction by Steve McConnell



Thank You